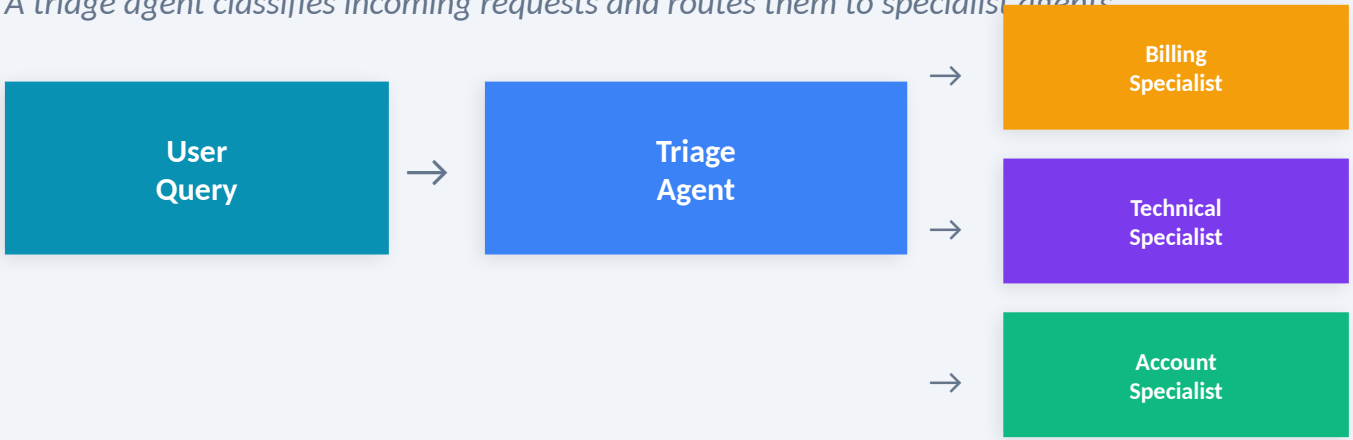
05

Advanced Patterns

Handoff · Magentic · Production Considerations

Handoff: Support Triage

A triage agent classifies incoming requests and routes them to specialist agents



When to Use

- Different input types need different expertise
- Keep specialists focused with lean prompts
- Routing logic = classification problem

Context: Structured HandoffContext

Structured handoff object (HandoffContext dataclass) passes context from triage to specialist. Contains: query, category, priority, extracted_info. Specialist does NOT see triage reasoning.

Handoff — Decision Guide

✓ When to Use

Different agents handle different stages of the workflow

Each stage requires specialized expertise

Responsibility needs to be clearly transferred

The next agent starts only when the previous one finishes

★ Why Choose It

Clean ownership boundaries between agents

Ideal for workflows with tiered expertise

Ensures each stage is handled by the right specialist

Reduces back-and-forth and rework

✗ When NOT to Use

The agent order is already known upfront (use Sequential)

One agent can complete the entire task

Current agent does not have enough context to perform handoff

➤ Example

Adaptive Product Issue Escalation

Feedback → Hardware Diagnostics → Software Telemetry → Risk Assessment → Root-Cause Specialist

Exercise: Support Triage Code

`exercises/07_handoff/01_support_triage.py`

Triage: parse() for Classification

```
class TriageDecision(BaseModel):
    category: str  # billing/technical/account
    priority: str  # low/medium/high
    reasoning: str
    extracted_info: dict  # order IDs, etc.

decision = client.chat.completions.parse(
    model=model,
    messages=[
```

Structured Handoff + Routing

```
@dataclass
class HandoffContext:
    customer_query: str
    category: str
    priority: str
    extracted_info: dict

# Specialist gets ONLY handoff context
handoff_text = (
    f"Query: {handoff.customer_query}\n"
```

Each Specialist Has Focused Tools

Billing	`lookup_order()`, `process_refund()`
Technical	`search_faq()`
Account	`search_faq()`

Magentic — Decision Guide

✓ When to Use

Agents should activate based on findings, not a fixed order

Workflow is adaptive — next step depends on discoveries

Tasks have conditional dependencies

Problem needs both structure and flexibility

★ Why Choose It

Smarter, context-aware flow — only necessary agents activate

Reduces wasted work (no unnecessary reviews)

Produces deeper reasoning without forcing a rigid path

Great balance between efficiency and thoroughness

✗ When NOT to Use

Solution path is deterministic and known upfront

Tasks are simple and don't need multi-round reasoning

Speed is critical and you can't afford planning overhead

Token budget is tight (manager + workers = many LLM calls)

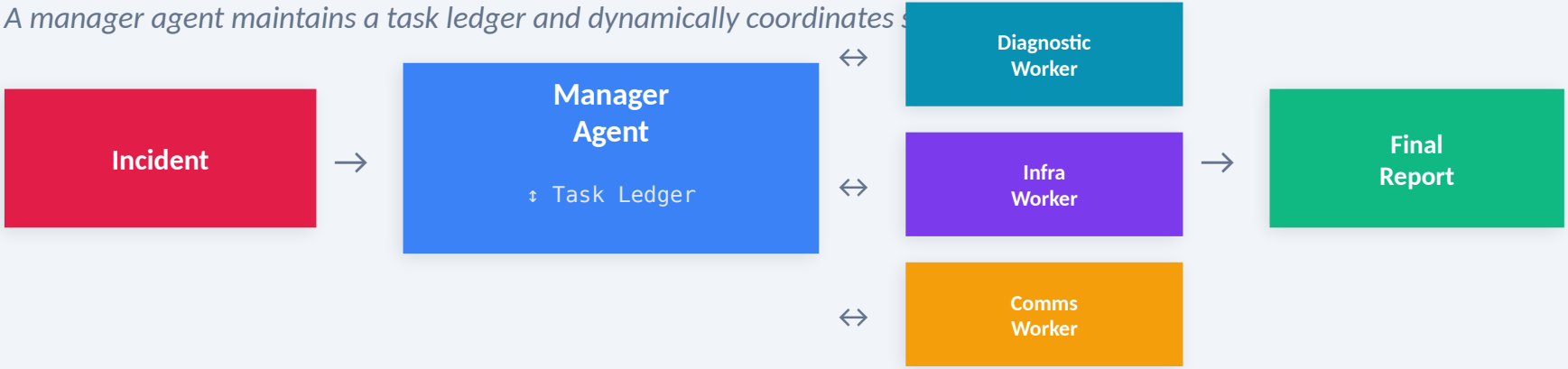
➤ Example

RFP Reviewer

Summary Agent → Risk Assessment → Compliance (only if risk found) → Final Decision

Magentic: Incident Response

A manager agent maintains a task ledger and dynamically coordinates s



When to Use

- Complex multi-step tasks with plan changes
- Intermediate findings may add new tasks
- Central coordinator tracks progress

Context: Task Ledger + Task-Specific

Workers get TASK-SPECIFIC context only — incident description + their specific task. The manager controls information flow and maintains the Task Ledger (shared mutable dataclass). Plan can adapt: new tasks added based on findings.

Exercise: Incident Response — 4 Phases

exercises/08_magentic/01_incident_response.py

Phase 1: Manager Creates Plan

```
class IncidentPlan(BaseModel):
    assessment: str
    tasks: list[PlannedTask]
```

Phase 2: Workers Execute Tasks

```
for task in ledger.tasks:
    # Worker gets ONLY task context
    messages = [
      {"role": "system", "content": PROMPT},
```

Phase 3: Manager Adapts Plan

```
class AdaptedPlan(BaseModel):
    analysis: str
    new_tasks: list[PlannedTask]
    ready_for_report: bool
```

Phase 4: Final Incident Report

```
# Manager synthesizes ALL findings
all_findings = "\n".join(
    f"Task #{t.id} ({t.assigned_to}): "
    f"{t.result}"
```

Task Ledger = shared mutable dataclass with tasks[], findings[], add_task(), complete_task(). The central coordination artifact.

Choosing the Right Pattern — At a Glance

Pattern	Decision Style	Use When	When NOT To Use
Sequential	Fixed, staged	Each step depends on previous; clear linear path	Steps not strictly ordered; agents need to collaborate
Concurrent	Parallel, independent	Tasks can run in parallel; need multiple perspectives	Tasks depend on each other; resource constraints
Group Chat	Collaborative dialogue	Multiple agents reason together; no single expert suffices	Need strict step-by-step; problem lacks clarity
Handoff	Expertise-based routing	Different inputs need different expertise; can't determine upfront	Flow is fixed; one agent can finish the job
Magentic	Adaptive, goal-driven	Plan must evolve based on outcomes; conditional dependencies	Solution path is deterministic; speed is critical

Choosing the Right Pattern

Can ONE agent handle it? — Yes → **Single Agent**

No ↓

Clear ordered stages? — Yes → **Sequential**

No ↓

Independent sub-tasks? — Yes → **Concurrent**

No ↓

Agents need to see each other? — Yes → **Group Chat**

No ↓

Input determines routing? — Yes → **Handoff**

No ↓

Magentic (adaptive planning)

Pro Tip

Start with Single Agent.

Before choosing a multi-agent pattern, ask: can I solve this with one agent that has good tools and a clear system prompt?

If yes, stop there.

Context Management Strategies

Full History

Single Agent, Group Chat

Pass entire conversation list. Simple but can hit token limits.

Fresh per Stage

Sequential

Each agent gets clean context with only previous output. Token-efficient.

Structured Objects

Handoff

Pass a typed dataclass (HandoffContext) between agents. Explicit and robust.

Task-Specific

Magentic

Manager curates what each worker sees — only incident context + specific task.

Context strategy is as important as choosing the right pattern.

Reliability & Human-in-the-Loop

Reliability Guardrails

⛔ **Max Iterations**
Every agent loop needs a hard limit

🔄 **Retry + Backoff**
Exponential backoff for transient errors

🛡️ **Tool Safety**
Return errors as tool results, don't crash

⏱️ **Timeouts**
Set at both request and agent levels

📉 **Graceful Degradation**
Partial results > total failure

Human-in-the-Loop

Approval Gates
Pause before irreversible actions (e.g. refunds > $100)

Escalation
Agent recognizes limits, routes to human expert

Feedback Loops
Human reviews agent output, provides corrections

Confirmation
Verify understanding before proceeding

Key Takeaways

1. An agent = LLM + Tools + Loop (Reason → Act → Observe)
2. Start with the simplest approach — don't over-engineer with multiple agents
3. 5 orchestration patterns cover the vast majority of multi-agent use cases
4. Context passing strategy is as important as choosing the right pattern
5. Reliability is non-negotiable — max iterations, retries, graceful degradation
6. Understanding patterns from scratch makes you a better user of any framework

Let's Build!

Start coding: exercises/00_setup/01_test_connection.py

OpenAI Docs

platform.openai.com/docs

MS Learn Patterns

learn.microsoft.com/azure/architecture/ai-ml/guide/ai-agent-design-patterns

Anthropic Agents

anthropic.com/engineering/building-effective-agents

Build from scratch. Understand the patterns. Choose any framework later.